

Date: Feb 23rd, 2026

Course Instructor(s)

Dr. Shakil Ahmed, Mr. Usama Antuley, Dr. Fahad Sherwani,
Ms. Anaum Hamid, Dr. Farrukh Salim Shaikh, Mr. Farooq Zaidi,
Ms. Abeeha Sattar, Ms. Alina Arshad, Ms. Javeria Farooq, Mr. Sandesh Kumar

Sessional-I Exam

Total Time (Hrs): 1

Total Marks: 15

Total Questions: 3

Roll No

Section

Student Signature

INSTRUCTION:

- Attempt all the questions in-order.
- Solve the problems as per duration of each question mentioned.

CLO #1: Summarize the notions of rational behavior and intelligent agents.

Question 01: Answer the following questions. (Not more than 3 lines)

[12 minutes]

[Marks: 4]

- Define admissible and consistent heuristics.
- Compare Space and Time complexity of Depth Limited and Iterative Deepening algorithms. Provide their Big O notations as well while comparing.
- A smart hospital ICU monitoring system continuously observes multiple patients using vital-sign sensors, automatically alerts doctors during emergencies, adjusts ventilator settings when required, and improves its decisions over time using past patient data. Describe the complete task environment of this agent using PEAS, classify the environment type, and identify the most appropriate agent type.
- What are the problems associated with Hill Climbing algorithm? Suggest its possible solutions.

CLO # 3: Apply methods of blind as well as informed search on the problem related to the programs.

Question 02: Part (a)

[15 minutes]

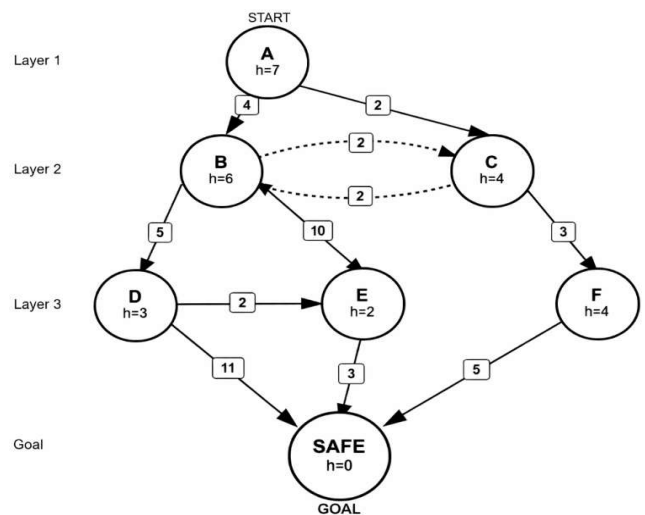
[Marks: 3]

- An intelligent evacuation system is deployed to help people reach a safe zone in a flood-prone region.
- Towns are connected via roads of different costs.
- Heuristic information (estimated distance to safe zone) is available.
- The system may be asked to start from any town and reach any safe zone.

Implement the following search strategies:

- Uniform Cost Search (UCS)
- Greedy Best-First Search
- Iterative Deepening DFS (IDS)

Results should record in the table below **after running each algorithm:**



Strategy	Write Path from S→G	Total Path Cost (S-G)	Number of Nodes Expanded to reach Goal	Memory Consumption (E.g., High, Medium, Low)
UCS				
Greedy Best-First Search				
IDS				

National University of Computer and Emerging Sciences

Karachi Campus

Question 02: Part (b)

[5 minutes]

[2 marks]

Based on the $f(n)$ values, select the **next state to expand** according to **A*** search. Path cost for each move = 1 unit. Heuristic function: Misplaced tiles $h(n)$ = number of tiles not in their goal position (blank not counted).

Initial State

1	2	3
4	—	6
7	8	5

Goal State

1	2	3
4	5	6
7	8	—

CLO # 3: Apply methods of blind as well as informed search on the problem related to the programs

Question 03: Part (a)

[10 minutes]

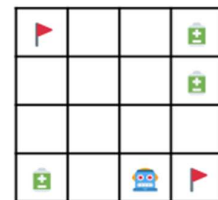
[Marks: 3]

a) Consider a delivery drone in figure 1 moving on a $N \times N$ grid.

- At each time step, the drone must move to an adjacent square (up, down, left, right). All actions cost 1.
- There are k fixed destination squares on the grid. The drone needs to visit all k destinations at least once.
- The drone starts with a battery charge of C units, and must always have at least 1 unit of charge at every time step. The drone loses 1 unit of charge per action.
- There are p power cells on the grid (with fixed locations). When the drone visits a power cell, its battery charge resets to C units, and the power cell cannot be used again.

- What is the maximum branching factor for this problem? Justify your answer in one brief statement.
- What is the size of the smallest state space representation for this problem? Provide justification in a brief statement.
- Enlist all the algorithms that are guaranteed to find the lowest cost solution to this problem.

Figure: Delivery Drone Grid



Sample grid with $N = 4$,
 $k = 2$ destinations 🚩,
 $p = 3$ power cells 🔋,
 and drone 🚖.

Question 03: Part (b)

[10 minutes]

[Marks: 3]

Solve the following problem using Genetic Algorithm. start with a population of n random strings. Suppose that String length $l = 10$ and population size $n = 6$. We toss a coin 60 times and get the following initial population.

S1=	1111010101
S2=	0111000101
S3=	1110110101
S4=	0100010011
S5=	1110111101
S6=	0100110000

Fitness Function Rule: Count 1's in each sample. Select S1, S3, S4 & S6 samples.

Crossover Rules:

- Apply crossover between S1 and S3 after 2nd gene.
- Apply crossover between S4 and S6 after 5th gene.

Mutation Rules:

- Apply mutation in updated S1 in the 7th gene
- Apply mutation in updated S3 in the 5th & 9th gene
- Apply mutation in updated S4 in the 6th & 9th gene
- Apply mutation in updated S6 in the 8th gene

After the above process, calculate the total fitness again as per the same rule and if gets increased, then perform one more iteration using the above same rules otherwise stop.

===== **Best of Luck** =====